

MR. LOBSTER

v3

Master Build & Vision Plan

The face of the AI revolution for local business.

Owner	Lukasz Bukowiecki
Date	30 April 2026
Version	v3.0 (synthesises v2 master, agentic addendum, future-proof analysis, Karpathy)
Build budget	4–6 hours / week
Build mode	Claude Code + Codex driven
First customer	TM Hughes & Son (Terry) — plumbing & heating, Chelmsford
Target	30–40 UK local SME tenants in 6 months

Supersedes: *Mr_Lobster_v2_Master_Build_Plan* (26 Apr) and *Mr_Lobster_v2_Update_Addendum_Agentic_Plan* (28 Apr). Where they conflict, this document is the single source of truth.

Contents

Part I — Strategic frame	
1. Executive summary	5
2. What changed in v3	6
3. The vision: Face of the AI revolution for local business	7
4. Software 3.0 reality check	8
Part II — Product	
5. Product architecture: agent-native from row 1	10
6. The three visible layers (Reception → Owner → Company Agent)	12
7. Memory & the learning loop (the moat)	14
8. Channels: voice, WhatsApp, future	16
9. The single configuration layer	17
Part III — The four foundations baked into v3.0	
10. Context engineering for voice + multi-turn	18
11. Evals & the golden dataset	20
12. MCP-first tool design	22
13. Outcome instrumentation (job value from day 1)	23
14. Provider abstraction everywhere	24
Part IV — Build methodology	
15. Claude Code + Codex operating model	25
16. Lukasz's actual job	26
17. The spec → code → test loop	27
Part V — Terry v3.0 launch (8 weeks)	
18. Week-by-week build sequence	28
19. Pre-go-live checklist & cutover	31
Part VI — 6-month roadmap	
20. v3.0 → v3.1 → v3.2 → v3.3	32
21. Customer acquisition rhythm alongside the build	33
Part VII — Stack, accounts, costs	
22. The exact account list	34
23. Stack rationale	35
24. Pricing & outcome-based hooks	36
25. Cost model per customer	37
Part VIII — Reliability, security, GDPR	
26. Reliability plan	38
27. Security & GDPR	39
Part IX — Risks & mitigations	40
Part X — Founder action plan	
28. Next 48 hours	41
29. Next 7 days & backlog	42
Closing — The face of AI revolution, locally	43

PART I

Strategic frame

The first four sections set the binding context. Everything that follows — architecture, build sequence, pricing, roadmap — is downstream of these decisions. Read this section carefully. Re-read it whenever a decision later in the document feels arbitrary.

1. Executive summary

Mr. Lobster v3 is one cloud-hosted, agent-native customer interaction platform for UK local service businesses. Voice and WhatsApp are channels of one product. Lead capture, escalation, owner notifications, business memory and (later) booking are features. The website sells one product, not three.

v3 supersedes v2 in three ways. First, it bakes the agent architecture in from the first commit, not as a v2.2 phase. Second, it locks in five engineering disciplines the v2 documents under-addressed: context engineering, evals, MCP-first tools, outcome instrumentation, and universal provider abstraction. Third, it accepts the honest timeline: at 4–6 hours per week, Terry v3.0 is an eight-week build, not a two-week sprint.

Three commitments this plan locks in

1. **Terry on v3.0 in 8 weeks.** Cloud-only stack. No site visit. Claude Code and Codex do the implementation; Lukasz reviews, decides, signs in.
2. **Self-serve onboarding by month 4.** Customer #4 provisions themselves through the website in under thirty minutes.
3. **Company Agent layer by month 6.** Memory, weekly digest, conversational configuration via WhatsApp, vertical defaults for plumbers and clinics.

Three things that change today

1. **Reprice and refold.** One product on the site (Mr. Lobster Reception Engine, evolving into Company Agent), one £299/month Standard tier, one £599 setup fee. Three-product framing dies before customer #2 signs.
2. **Start Meta WABA verification immediately.** The 5–10 day clock runs in parallel with the build. By go-live week the verification is done.
3. **Open the build accounts.** Render, Postgres, Clerk, Stripe, Twilio, ElevenLabs, Anthropic, Sentry, Axiom, UptimeRobot. List in §22. Lukasz signs up; Claude Code wires them.

BLUNT NOTE

v1 didn't fail because of bad code. It failed because it was a personal toolkit dressed as a product. v2 was right to throw it away. v3 is right to throw away v2's *14-day Terry sprint*. At your real availability that timeline produced a shipped product on paper and zero hours for outbound, content, or the VFX database. Eight weeks with Claude Code carrying the implementation is the realistic path. Discipline lives in what you cut, not in how fast you sprint.

2. What changed in v3

Five categories of change. None of these are cosmetic. Each one closes a gap that would have hurt Terry, slowed customers 2–10, or made v3.2 impossible to build without a rewrite.

2.1 Timeline made honest

v2 master plan promised Terry live in 14 days. Realistic at 4–6 hours/week is 8 weeks for v3.0 minimal launch, with weeks 9–10 as hardening. The addendum already flagged this; v3 commits to it. Outbound continues at half-cadence during the build, full cadence from week 9.

2.2 Agent architecture moved forward

v2 sequenced the agent layer as v2.2 (~6 months out). v3 puts the agent foundation in v3.0. That doesn't mean shipping the Company Agent on day 56; it means the *plumbing* for it (typed events, tool-call logs, memory tables, audit trail, MCP wrappers, provider abstraction) is in place from row one. The v2.2 reveal then becomes a UI and packaging exercise, not a re-architecture.

2.3 Five engineering disciplines made non-negotiable

Surfaced by the future-proof analysis. Detailed in Part III. Headline:

- **Context engineering** — explicit window-management strategy for voice. Voice calls fail at turn 8, not turn 1.
- **Evals + golden dataset** — 50 hand-labelled call transcripts before Terry goes live. Without it, every prompt edit is a guess.
- **MCP-first tools** — every tool is an MCP server from day one, not an internal function. Cost is minimal; the v3.1 owner agent connects to them for free.
- **Outcome instrumentation** — every captured lead carries an estimated job value. Without this, you cannot price on outcomes, and you cannot tell Terry what Mr. Lobster saved him.
- **Provider abstraction everywhere** — wrap Anthropic, Twilio, Meta, ElevenLabs, Google Calendar behind interfaces. Six months from now Sonnet 5 ships and you swap one line.

2.4 Build methodology designed around Claude Code + Codex

The v2 documents assumed Lukasz would write most of the code in evening sessions. v3 assumes Claude Code (and a parallel Codex agent for narrow tasks) writes the code; Lukasz writes the spec, opens the accounts, reviews PRs, runs the test calls. This shifts the bottleneck from typing to deciding — which matches your weekly budget and your strengths. Detailed in Part IV.

2.5 Strategic positioning sharpened

v2 sold 'missed-call protection'. Right product, partial framing. v3 sells the *same outcome today* but reveals the *company agent identity* on a deliberate schedule. By month 6 Mr. Lobster is the local face of AI for SMEs in Essex and London, not a chatbot vendor. Detailed in §3.

WHAT DID NOT CHANGE

The product vision, the architecture spine (one backend, one DB, one queue, many channels), the UK local-trade wedge, Terry first, capture-only as the safe default, the £299 Standard tier, the death of OpenClaw and WhatsApp Web in any production path. v3 keeps everything that was right in v2.

3. The vision: Face of the AI revolution for local business

Karpathy makes a counter-intuitive observation in *Software is changing again*. Historically, transformative technologies — electricity, computing, GPS, the internet — diffused from governments and large corporations down to consumers. AI has flipped that. The first heavy users of LLMs are individuals asking how to boil an egg. Governments and large enterprises are lagging.

UK local SMEs are also lagging — but for a different reason. They're lagging because no one has built the AI tool that fits how they actually work. They're under sinks, in vans, with patients, on Friday nights. They don't want a SaaS dashboard. They don't want to read documentation. They want the work handled.

VISION

Mr. Lobster is the company agent for UK local service businesses. It answers, remembers, follows the rules, knows the customers, advises the owner, and improves the front office every week it operates. By month six, when the rest of the small-business world is still saying 'we should look into AI', our customers will already be running on it.

3.1 What 'face of the revolution' means in practice

Three things, in this order:

- 1. The product is invisible until it isn't.** Terry doesn't feel like he bought AI software. He feels like he hired a receptionist who never sleeps. The AI identity is underneath — visible to him only when he wants to see it.
- 2. The owner's relationship is conversational.** The owner doesn't log into a dashboard to change a price; he sends a WhatsApp to Mr. Lobster HQ. By the time competitors ship a chatbot, our customers are already operating their business through one.
- 3. The moat is institutional memory.** Six months in, Mr. Lobster knows Terry's business better than any new tool a competitor could ship. Switching cost is not 'set up another bot.' It's 'rebuild the institutional memory of my front office.'

3.2 Why local SME is the right wedge

Three structural reasons make this market the right place to be the local face of AI right now:

- **The pain is acute and quantifiable.** A missed call from a boiler emergency is £200–600 of lost work. A missed enquiry at a multi-practitioner clinic is a £1,500 booking. These are not optimisation problems. They are revenue protection problems.
- **The buyer is the user.** No procurement. No IT department. No six-month sales cycle. The plumber is the decision-maker, the user, and the budget holder.
- **The competitive landscape is empty.** WhatsApp automation tools exist. Phone-answering services exist. Nobody has built the integrated agent layer designed for this exact buyer. The local trades are the last segment large incumbents will reach because the customer acquisition cost looks unattractive at scale.

3.3 The narrative arc — three eras of one customer

Month 1. Terry buys Mr. Lobster because his phone rings when he's under a sink. Mr. Lobster answers, takes the details, texts him within 30 seconds. He stops losing jobs.

Month 3. Terry notices Mr. Lobster recognises returning customers. He starts asking it questions on WhatsApp: 'who needs a callback today?' He starts updating prices by sending it a message.

Month 6. Mr. Lobster sends Terry a Monday digest: 'Last week you had 47 calls. 12 were after 6pm. Boiler service enquiries up 40% this month. Three customers from October haven't been contacted in 90 days — want me to message them about winter checks?' Terry says yes by replying to a WhatsApp. He has not opened a dashboard once this month.

That arc is the product. It's also the marketing story. It's also the moat. Every architecture decision in this document either supports that arc or doesn't ship.

4. Software 3.0 reality check

Karpathy's talk reframes what we're building. Worth internalising before any code is written, because the framing changes how you weigh trade-offs.

4.1 The three software paradigms

Paradigm	What it is	Where it shows up in Mr. Lobster
Software 1.0	Code we write — TypeScript, SQL, queue logic, webhook handlers.	The harness. The thing that ships even if the model gets dumber. ~70% of v3.0.
Software 2.0	Neural network weights — the LLMs themselves, the voice models, the Whisper transcription model.	Used as services. We don't train these. We rent them. Wrapped behind providers.
Software 3.0	Prompts as programs. SOUL.md, agent instructions, capture-field schemas, tool descriptions.	Composed from data, not edited by hand. Versioned. Tested with evals. The thing that gets calibrated weekly.

All three coexist in v3. The mistake to avoid is treating any of them as 'the important one.' The harness is what ships reliably. The model is what handles ambiguity. The prompt is what encodes the business.

4.2 Five Karpathy-grade truths the build respects

- 1. The harness does more work than the model.** Swap Sonnet for Sonnet 5 and the system still ships. Swap the queue, the audit trail, or the provider abstraction and you have outages. Build the harness like it matters more than the model — because it does.
- 2. Context windows are working memory, not long-term memory.** Treat every voice conversation as a stateless agent that must reload the relevant tenant context at the start of every call and prune ruthlessly during. (See §10.)
- 3. Build for agents, not just for humans.** The dashboard is read-only for v3.0; structured queries against tenant data are first-class from row one. When v3.1 ships an owner agent that answers 'who needs a callback today?' on WhatsApp, the data is already accessible.
- 4. The autonomy slider is a real product axis.** Capture-only ships first. Booking is next. Fully autonomous follow-up is last. Each step is the owner choosing how much to let go. The product makes that slider visible and explicit.
- 5. LLMs are gullible, jagged, lossy.** Prompt injection from a caller is a real attack surface. The agent must quote prices from a structured table, not improvise. It must escalate on triggers it can match against a list, not on vibes. Constraints are features.

WHY THIS SECTION EXISTS

Because in eight weeks, when Claude Code asks 'should we just hard-code the prompt into the agent file or load it from Postgres?', the answer is informed by §4. Prompts are programs. Programs live in version control or in a database, not in a JSON file edited by hand. v1's biggest error was treating prompts as configuration. v3 treats them as code.

PART II

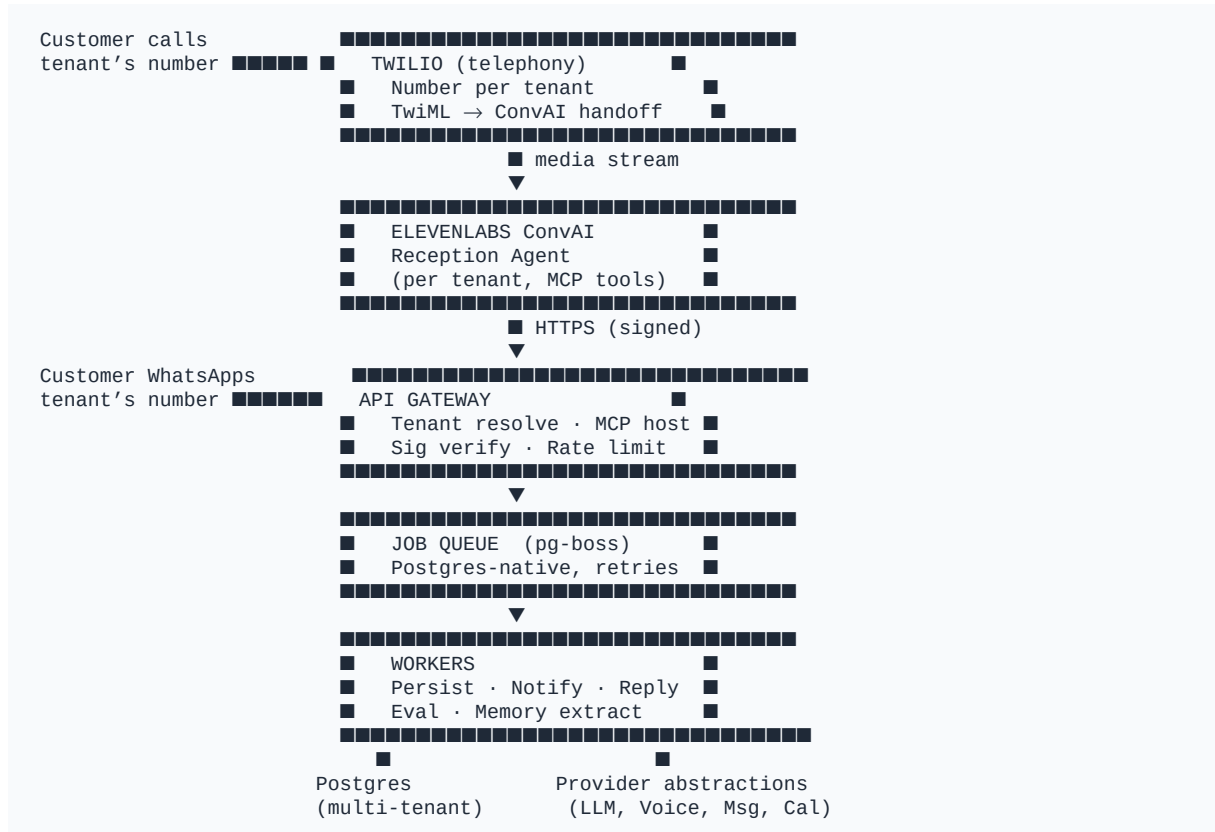
Product

What gets built. What it does. How it's structured. This part defines the product the customer experiences and the architecture underneath it. The five engineering disciplines (Part III) are how we make sure it actually works.

5. Product architecture: agent-native from row 1

One backend. One database. One queue. Multiple channels. Agents are first-class primitives, not a v2.2 retrofit.

5.1 The architecture diagram



5.2 Why this shape, and what's different from v2

- **Async spine.** Same as v2 — every cross-system call goes through the queue. v1's synchronous chain is what killed it.
- **One backend, multi-tenant from row 1.** Same as v2 — tenant_id column with Row Level Security. No per-customer servers, ever.
- **Tools are MCP servers.** This is new. log_enquiry, get_business_rules, get_service_catalogue and friends are MCP servers behind the gateway. Reception agent calls them via MCP. v3.1 owner agent connects to the same MCP host. (Detailed in §12.)
- **Provider interfaces.** This is new. LLMProvider, VoiceProvider, MessagingProvider, CalendarProvider — wrapper interfaces from day 1. Concrete implementations behind them. (§14.)
- **Eval worker in the queue.** This is new. Every captured lead is also queued for eval scoring against the golden dataset. Failed eval scores feed a Monday review. (§11.)
- **Memory extraction worker.** This is new. Every conversation is post-processed into structured memory updates (services aliases, FAQs, escalation refinements). All gated behind owner approval. (§7.)

5.3 Database — core tables for v3.0

Identity & config	Conversation	Operations & agents
tenants	conversations	agent_events
users	messages	agent_tool_calls
channels	calls	queue_jobs (pg-boss)
phone_numbers	leads (with job_value_estimate)	audit_logs
whatsapp_numbers	lead_outcomes	eval_runs
business_profiles	customers	memory_proposals
service_catalogue	customer_preferences	memory_corrections
pricing_rules	conversation_summaries	usage_records
capture_fields	extracted_signals	billing_subscriptions
escalation_rules	unanswered_questions	

WHAT'S NEW IN THE SCHEMA

job_value_estimate on every lead — outcome instrumentation (§13). **eval_runs** table — golden-dataset scoring history (§11). **memory_proposals / memory_corrections** — agent-suggested changes always require owner approval before commit (§7). **extracted_signals / unanswered_questions** — every conversation produces typed signals for the learning loop, not just a transcript blob.

6. The three visible layers

What the customer sees, in sequence. Architecture stays the same; the visible product evolves.

Layer	Phase	What the owner experiences
L1 — Reception Agent	v3.0 (weeks 1–8)	The phone gets answered when the owner can't. WhatsApp lands within 30s of every missed call. Every captured lead has a structured job-value estimate.
L2 — Owner Assistant Agent	v3.1 (months 3–4)	Owner sends WhatsApp questions to Mr. Lobster HQ: 'who needs a callback?', 'how many calls did I miss last week?', 'boiler service is £90 from Monday.' Confirmed changes commit immediately.
L3 — Company Agent	v3.2 (months 5–6)	Monday morning digest. Proactive follow-up suggestions. Returning customer recognition. Pricing-gap detection. Weekly insights. The system advises, the owner approves, the system commits.
L4 — Vertical Company Agents	v3.3 (post-month 6)	Plumber Agent, Heating Engineer Agent, Clinic Reception Agent — vertical defaults, trade vocabulary, compliance constraints. Same platform, different prepackaged personality and tools.

6.1 L1 — Reception Agent (v3.0)

What ships in v3.0 is deliberately narrow. Doing this well is more important than doing more.

What it does:

- Answers calls when the owner doesn't pick up (BACKUP mode, 20s ring delay)
- Identifies caller intent within 2–3 turns
- Quotes prices from the structured services table — never improvises
- Captures structured lead fields (name, phone, postcode, job_type, urgency, notes)
- Estimates job value at capture time and writes it to the lead
- Detects urgent triggers in real time and fires owner alert before call ends
- Owner gets WhatsApp within 30s, summarised and structured

What it explicitly does not do in v3.0:

- Inbound WhatsApp conversations from customers (v3.1)
- Calendar booking (v3.1, owner-toggled)
- Returning customer recognition (v3.1)
- Conversational config from owner (v3.2)
- Self-serve onboarding (v3.1)

6.2 L2 — Owner Assistant Agent (v3.1, months 3–4)

The first time the owner experiences Mr. Lobster as 'an agent that knows my business' rather than 'an answering service'. Five command types:

- **Pricing.** 'Boiler service is £90 from Monday.'
- **Rules.** 'Don't take jobs outside Essex.'

- **Scheduling.** 'I'm on holiday 5–12 May.'
- **Queries.** 'How many calls did I miss this week?'
- **Follow-up.** 'Remind me tomorrow to call back Mrs Patel.'

Every command is parsed → confirmed → committed → audit-logged with the natural-language source. Owner never edits a config screen.

6.3 L3 — Company Agent (v3.2, months 5–6)

Where Mr. Lobster becomes the local face of AI. The Memory Agent and Growth Advisor Agent run in the background; the owner experiences this as a Monday WhatsApp digest:

"Morning Terry. Last week: 47 calls handled, 12 after hours. Boiler service enquiries up 40% this month — first cold snap. Three customers from October haven't been contacted in 90 days, total estimated value £840. Want me to draft a winter check-up message? Reply YES to send."

DESIGN CONSTRAINT

The Company Agent **proposes**; the owner **approves**. The agent never invents business rules or commits material changes without confirmation. This is what makes it trustworthy enough to operate a real business — and it's the discipline that separates a credible AI front-office from a chatbot.

7. Memory & the learning loop (the moat)

A WhatsApp bot is not defensible. The accumulated, structured, queryable, continuously-improving knowledge of how each business actually runs is. This section is the moat.

7.1 Five layers of memory

Layer	Contents	Lifecycle
Business memory	Services, prices, hours, areas, exclusions, credentials, tone/persona.	Seeded at onboarding. Edited via wizard or (v3.2) WhatsApp commands.
Customer memory	Names, phone numbers, WhatsApp IDs, addresses, preferences, past jobs.	Created on first interaction. Updated post-call. Surfaced to AI on next contact.
Conversation memory	Transcripts, structured summaries, extracted intents, unresolved questions.	Written by post-call worker. Pruned / summarised after 7 days.
Operational memory	Missed-call patterns, response times, conversion rates, alert delivery stats.	Aggregated nightly. Feeds Monday digest.
Learning memory	Owner corrections, approved memory updates, rejected suggestions, confidence scores.	Append-only. Audit trail for every prompt change.

7.2 The learning loop

Four input streams feed memory; one read path uses it.

Stream 1 — External ingestion (v3.2)

At onboarding (v3.2 self-build feature), the system scrapes the tenant's website, Google Business Profile, and Google Reviews. Output: a fully populated tenant config — services, hours, area, tone profile, capture fields — before the owner answers a single wizard question. Approval interface is a WhatsApp conversation, not a form.

Stream 2 — Live conversation signals (v3.0+)

Every call transcript and WhatsApp thread is post-processed by an extraction worker. Outputs typed signals: new questions the agent couldn't answer, job types not in the services table, service aliases, urgency patterns, incomplete captures. Each signal becomes a row in `extracted_signals`. The owner reviews the queue weekly (or via WhatsApp from v3.1).

Stream 3 — Owner decisions and corrections (v3.1+)

Every owner action is a teaching signal: marking a lead as junk, correcting an AI response, sending a WhatsApp command. Each one writes to `memory_corrections` with timestamp and natural-language source. The audit log doubles as a training record.

Stream 4 — Nightly intelligence worker (v3.2)

Reconciles call logs, summarises the day, calibrates the SOUL against the current structured tables (catches drift), generates the Monday digest. Detailed in §10 (context) and §11 (evals).

7.3 Conversation-time read path

What the AI actually queries when a call starts. This is the gap v2 didn't close.

1. Tenant context loaded once per session (cached):
persona · services · rules · capture_fields · tone
2. Customer recognised? (inbound number / WA ID match)
YES → load name, address, past_jobs, preferences
NO → start new customer record on capture
3. Mid-conversation FAQ lookup (on-demand):
semantic search against faqs table
confidence > 0.85 → answer
no match → log unanswered_question, callback offer
4. Escalation check (every conversational turn):
match against escalation_rules table
triggered → mark urgent, alert owner NOW
5. Call ends → post-call worker:
persist lead with job_value_estimate
extract signals (unanswered Qs, aliases, gaps)
update customer record
queue eval scoring against golden dataset

WHAT GOES STATIC vs DYNAMIC

SOUL.md (persona, tone, voice instructions) is semi-static — regenerated on config change or weekly calibration, not per call. **Services, rules, FAQs, customer records, escalation triggers** are dynamic — read fresh per conversation. This means a price change at 9am affects the 9:05am call, but the AI's personality stays consistent.

8. Channels: voice, WhatsApp, future

Voice and WhatsApp are channels of one engine, not separate products. Same SOUL, same services table, same rules, same capture fields, same lead schema. The owner sees one Inbox regardless of where the lead came from.

8.1 Voice (v3.0)

- **Provider:** ElevenLabs ConvAI, wrapped behind VoiceProvider interface (§14).
- **Telephony:** Twilio Programmable Voice. Mr. Lobster owns the number; tenant's existing number forwards (or new number issued for signage).
- **Mode:** BACKUP — Twilio rings owner first 20s, hands to AI on no-answer. ALWAYS mode available as a config option.
- **Latency budget:** tool calls must return in <1.5s p95. If a call exceeds this, the agent fills the silence with a holding phrase. (Detailed in §10.)
- **Failover:** websocket failure detected in 2s → voicemail recorded → Whisper transcription → surfaced as a normal lead.

8.2 WhatsApp — phased migration

Phase	WhatsApp transport	Why
Terry v3.0 (weeks 1–8)	Twilio WhatsApp for owner alerts only.	Faster to ship. No template approval needed for service-conversation alerts within owner-initiated 24h window. No inbound customer conversations yet.
v3.1 (month 3 onward)	Meta Cloud API direct for new tenants. Inbound customer conversations enabled.	WABA verification (started day 1) is complete. Half the cost per conversation. Direct feature pipeline from Meta.
v3.2 (month 6)	All tenants on Meta Cloud API. Twilio WhatsApp retired.	Migration is invisible — Terry's number stays the same; the gateway switches send/receive endpoints.

8.3 Future channels (v3.3+)

The MessagingProvider interface accommodates these without architectural change:

- SMS as a third channel (Twilio).
- Web chat widget for tenants' websites.
- Instagram DM (Meta Business API).
- Email enquiry capture (forwarding rules).

EXPLICITLY KILLED — DOES NOT RETURN

OpenClaw or any tool in any production path. WhatsApp Web / QR pairing / browser-emulating transports of any kind. Telegram bots as ops infrastructure. Mac Mini fallback compute. Tailscale across production. SSH-as-deploy. Per-customer servers. v1's ghosts stay buried.

9. The single configuration layer

v1's biggest hidden cost was configuration drift across four places: data.json, ElevenLabs dashboard, OpenClaw config, hand-edited prompt files. v3 has one source of truth: Postgres rows.

The SOUL.md is **generated** from these rows at provisioning and on calibration. It is never hand-edited. It is never the source of truth — the rows are.

9.1 What composes the SOUL

Source row	Contributes to SOUL	Edited via
business_profiles	Business name, hours, area, credentials.	Wizard (v3.0). WhatsApp command (v3.2).
service_catalogue	Structured price list. AI quotes ONLY from this.	Wizard (v3.0). WhatsApp command (v3.2).
pricing_rules	Weekend rates, emergency surcharges, fixed vs by-quote.	Wizard (v3.0). WhatsApp command (v3.2).
capture_fields	What the agent must collect before ending a conversation.	Wizard (v3.0).
escalation_rules	Trigger keywords/conditions, alert routing.	Wizard (v3.0). WhatsApp command (v3.2).
persona (sliders)	Tone, pace, formality, voice ID. Composes phrasing rules.	Wizard (v3.0).

9.2 The six-step wizard (v3.1)

v3.0 onboarding is admin-triggered (Lukasz runs a script) for Terry and customers 2–3. v3.1 ships a self-serve wizard, target 15 minutes:

- 1. Business basics** (2 min) — name, postcode (auto-timezone), owner mobile, trade, logo, emergency number.
- 2. Personality** (3 min) — sliders: Formal↔Friendly, Quick↔Patient, Brief↔Detailed. Voice dropdown (6 pre-screened British voices).
- 3. Services & pricing** (5 min) — table editor. Each row = a structured price. AI quotes from this row, not from improvisation.
- 4. Business rules** (2 min) — hours, areas, 'what we don't do' (becomes negative constraints in SOUL).
- 5. Escalation** (1 min) — checkboxes for urgent triggers. Backup engineer redirect.
- 6. Channels** (2 min) — Twilio number provisioned (Mr. Lobster picks area code by postcode). WhatsApp Meta direct. Mode (BACKUP / ALWAYS).

'Save and go live' — what happens in 90 seconds:

1. Insert tenant + child rows.
2. Compose SOUL.md from sliders + services + rules.
3. Provision ElevenLabs agent via API with the SOUL and the MCP tool list.
4. Provision Twilio number, attach TwiML pointing to ConvAI handoff.

5. Send synthetic test call to confirm end-to-end.
6. Mark tenant status = LIVE.

PART III

Five foundations baked into v3.0

Identified by the future-proof analysis as the engineering disciplines that separate teams that ship reliable agents from teams that don't. v2 deferred most of these. v3 builds them into the foundation.

Cost: minimal — these are mostly discipline, not features. Benefit: the v3.1 and v3.2 milestones become weeks of work, not months of rewrite.

10. Context engineering for voice + multi-turn

Karpathy's framing: the context window is working memory. A voice agent that loses track of its goal at turn 8 is not a model problem; it's a context engineering problem. v3 has an explicit strategy.

10.1 The four context layers, in order

Layer	What it contains	When loaded	Token budget
L1 System	Static persona, tone, behavioural rules, MCP tool descriptions.	Once at session start.	~1,500
L2 Tenant context	Compiled summary of services, prices, rules, hours, area, escalation triggers.	Once at session start, refreshed if >5 min old.	~1,000
L3 Customer context	If recognised: name, address, last 3 jobs, preferences. Else empty.	Once at session start (post-recognition).	~400
L4 Conversation	Recent turns (sliding window) + running structured state (captured fields, outstanding fields, escalation flag).	Updated every turn. Pruned at threshold.	~2,000 max

10.2 Pruning strategy (the rule)

Voice conversation state is divided into **structured slots** and **conversational history**. Slots persist; history is prunable.

Structured slots are written every turn the agent extracts a value: customer name, phone, postcode, job_type, urgency, captured_notes, outstanding_questions. These *never* leave the context until the call ends.

Conversational history (raw turns) sits in a sliding window. Once the window exceeds a threshold (~12 turns or ~1,500 tokens), older turns are summarised by a synchronous Haiku call into a 1–2 sentence rolling summary that replaces them. The structured slots remain intact.

WHY THIS MATTERS

Without pruning, by turn 8 a complex boiler enquiry buries the original capture goal under tool outputs and clarifications. With this strategy, the agent at turn 12 still knows: caller is Sarah Patel at CM2 4DT, job is 'boiler dripping cold water', urgency is amber, postcode confirmed, callback number outstanding. The conversation feels natural; the capture is reliable.

10.3 Latency budget and tool-call discipline

Voice conversations live or die on latency. The v3 budget:

Operation	Target p50	Target p95	What happens if exceeded
Tool call (any MCP)	<500ms	<1.5s	Agent fills with a holding phrase ("just checking that for you").
LLM turn (ElevenLabs ConvAI)	<800ms	<2s	Native to ElevenLabs; hard cap at 4 min total call duration.
End-to-end webhook to DB write	<200ms	<1s	Returns 200 immediately; persistence on the queue.

Tool design rule: any MCP tool the Reception Agent calls during a live voice conversation must be designed for <1.5s p95. If a tool needs to do real work (e.g. semantic FAQ search, customer lookup), it's pre-warmed at session start, not hit cold mid-call.

10.4 The unanswered question protocol

When a caller asks something the agent can't confidently answer (no FAQ match, not in services table, ambiguous):

1. Agent does not improvise. It does not hallucinate.
2. Agent acknowledges, captures the question into `outstanding_questions`, and offers a callback: 'That's a good question — let me get Terry to call you back on that. Best number?'
3. Question is logged to `unanswered_questions` table with the lead.
4. Owner sees it in the lead detail and (v3.1) on weekly digest: 'Three customers asked about underfloor heating last week — want me to add it?'

11. Evals & the golden dataset

PRINCIPLE

You cannot improve what you cannot measure. Sentry tells you when the system is broken. Evals tell you when the AI is doing a poor job. These are different problems. Without evals, every prompt change is a guess.

11.1 The golden dataset

Before Terry goes live, build a golden dataset of **50 hand-labelled call transcripts**. An afternoon's work. Highest leverage activity in the entire 8-week build.

Source the 50 examples from:

- Terry's actual call recordings (with consent — pre-Mr.-Lobster baseline). 30–40 examples.
- Synthetic adversarial cases — caller is angry, caller has thick accent, caller is rambling, caller asks about a service not in the table, caller mentions gas smell halfway through. 10–20 examples.

Each example is labelled with the **ground-truth lead** the agent should produce:

```
name: Sarah Patel
phone: 07700 900123
postcode: CM2 4DT
job_type: boiler_repair
urgency: amber
captured_notes: "dripping cold water from below"
job_value_estimate: £180-280
escalation_triggered: false
```

11.2 The eval harness

Every captured lead in production runs through an eval worker. Two layers:

Layer 1 — Field-level structural eval (cheap, deterministic)

- Did the agent capture all required fields?
- Is the phone number a valid UK format?
- Is the postcode a valid UK format?
- Is job_type drawn from the tenant's services table (or correctly tagged as 'other')?
- Is urgency one of the allowed values?
- Did escalation fire if escalation triggers were present in the transcript?

Layer 2 — LLM-as-judge eval (expensive, qualitative)

- Sample 10% of production leads + all golden dataset examples.
- Haiku judges: did the agent stay on tone? Did it improvise a price? Did it miss an obvious escalation cue? Did it prematurely end the call?
- Score 1–5 per dimension. Weekly aggregated to dashboard.

11.3 Regression discipline

Every SOUL change, every tool description change, every prompt edit triggers an automatic eval run against the golden dataset. If field-level pass rate drops, the deployment is blocked. If LLM-as-judge scores drop >0.5 average, Lukasz is paged for review.

v3.0 ABSOLUTE MINIMUM

Field-level eval running against golden dataset of 50 examples, blocking deploys on regression, dashboard view of pass rate over time. LLM-as-judge can wait until v3.1 (month 3) — but the golden dataset itself must exist before Terry takes a single live call. This is the one item in v3.0 that is genuinely non-negotiable.

11.4 What the eval dashboard shows the owner

Tenants don't see eval scores directly — but the metric they do see (lead completeness, escalation accuracy) is rolled up from evals. By v3.2, the Monday digest includes: 'Mr. Lobster captured 47 leads last week. 45 were complete. 2 needed manual completion — Terry, want to teach Mr. Lobster what to ask there?'

12. MCP-first tool design

Every tool the system exposes is an MCP server from day one. Not a v3.2 retrofit. The overhead of writing a tool as an MCP server vs. an internal function is hours, not weeks. The future-proofing is enormous.

12.1 The v3.0 tool list

MCP server	Used by	Phase
log_enquiry	Reception Agent (voice)	v3.0
get_business_rules	Reception Agent	v3.0
get_service_catalogue	Reception Agent	v3.0
search_faq	Reception Agent (mid-conversation)	v3.0
check_escalation_triggers	Reception Agent (every turn)	v3.0
estimate_job_value	Reception Agent (at lead capture)	v3.0
create_lead	Internal worker	v3.0
notify_owner	Internal worker	v3.0
send_sms_fallback	Internal worker (on Meta failure)	v3.0
score_eval	Eval worker	v3.0
get_customer_profile	Reception Agent (v3.1) + Owner Agent (v3.1)	v3.1
search_leads	Owner Agent	v3.1
update_lead_status	Owner Agent	v3.1
create_follow_up	Owner Agent	v3.1
propose_memory_update	Memory Agent (v3.2)	v3.2
approve_memory_update	Owner Agent (v3.2)	v3.2
update_business_rule	Owner Agent (v3.2)	v3.2
get_calendar_availability	Booking flow (v3.1)	v3.1
create_booking	Booking flow (v3.1)	v3.1

12.2 Why MCP and not internal functions

- **v3.1 owner agent gets them for free.** When the WhatsApp HQ owner agent ships, it just connects to the same MCP host. No rewriting.
- **External agent integration becomes possible.** Tenants who want to plug Mr. Lobster into their own AI workflow (rare today, common in 18 months) can — through MCP. We don't need to ship a public API.
- **Provider-agnostic.** If we swap from ElevenLabs ConvAI to Twilio ConversationRelay, the tools come along unchanged.

- **Auditable per-call.** Every tool call is logged with inputs, outputs, latency, and the calling agent. agent_tool_calls table populates from the MCP layer for free.
- **Karpathy’s ‘harness over model’ principle.** Tools are the harness. MCP is the modern, standard way to write that harness.

12.3 Permission model

Tool category	Who can call	Approval needed
Read business knowledge	All agents	No
Create lead / notify owner	Reception Agent only	No (it's the job)
Update lead status / create follow-up	Owner Agent (after owner confirmation)	Implicit via WhatsApp confirmation
Update pricing / rules	Owner Agent only	Explicit owner confirmation on every change
Approve memory update	Owner only (via WhatsApp)	Explicit owner confirmation

DESIGN CONSTRAINT

Material changes (prices, rules, escalation triggers) **always** require owner confirmation before commit. The Memory Agent **proposes**; only the owner's explicit approval commits. This is what makes the system safe to operate a real business — and it's legally defensible if anything goes wrong.

13. Outcome instrumentation

Without this, you can never tell Terry what Mr. Lobster saved him. You can never move to outcome-based pricing. And you give up the strongest single retention argument: the monthly invoice that comes with a number like ‘£8,400 of revenue protected.’

13.1 job_value_estimate — at lead capture

Every lead captured carries a job_value_estimate. Three components, populated by the estimate_job_value MCP tool at capture time:

Field	Type	How populated
estimate_low	Currency	Tenant's services_catalogue entry for matched job_type, low end.
estimate_high	Currency	Tenant's services_catalogue entry, high end. For ‘by quote’ jobs, tenant-provided default range.
confidence	0–1 score	1.0 = exact match to a catalogue row. 0.5 = inferred from job_type. 0.2 = wholly inferred. Below 0.5, the lead is flagged for owner review.

13.2 lead_outcomes — closing the loop

Each captured lead has a status that the owner updates (or the system infers from calendar/booking integration in v3.1+):

Status	How set	Effect
captured	Default at lead creation	Counts toward ‘leads protected’ metric.
contacted	Owner marks in dashboard / replies to alert ‘done’	Closes the open-loop on the alert.
booked	Owner marks, OR system sees calendar event linked	Counts toward ‘jobs booked’ metric. job_value_estimate counted as protected.
completed	Owner marks (or v3.2 weekly nudge)	Locks in the actual job value.
lost	Owner marks	Counts toward conversion metrics.
junk	Owner marks	Excluded from metrics. Pattern fed to the spam filter.

13.3 What the owner sees

Monthly invoice (v3.0) and Monday digest (v3.2) carry the headline metric:

***This month, Mr. Lobster handled 142 calls when you couldn’t.
Captured 89 leads · Booked 34 jobs · Estimated revenue protected: £8,420***

13.4 Why this enables outcome-based pricing

With three months of outcome data per tenant, two pricing variants become possible:

- **Performance tier:** £99/month base + 3% of estimated revenue protected, capped at £599. Lower friction to sign up; aligned incentives.
- **Premium tier:** £499/month flat + Company Agent. The customers who get to here will not switch.

v3 doesn't commit to either pricing model today — that's a v3.2 decision made with real cohort data. But the instrumentation that lets you decide must exist from day one.

14. Provider abstraction everywhere

The v2 master plan introduced a VoiceProvider interface. v3 extends this discipline to every external dependency. The cost is one extra file per provider; the benefit is vendor-independence as the LLM landscape churns.

14.1 The five interfaces

Interface	v3.0 implementation	Future swap candidates
LLMProvider	Anthropic (Claude Sonnet for replies, Haiku for routing & eval).	OpenAI, Google Gemini, future Anthropic models.
VoiceProvider	ElevenLabs ConvAI.	Twilio ConversationRelay, OpenAI Realtime API, Vapi.
MessagingProvider	Twilio WhatsApp (Terry, weeks 1–8). Meta Cloud API (from v3.1).	Vonage, MessageBird, native channel SDKs (Instagram DM, web chat).
TelephonyProvider	Twilio Programmable Voice.	Vonage, SignalWire, Telnyx.
CalendarProvider	Not in v3.0. Google Calendar in v3.1.	Microsoft 365, Calendly, Cal.com.

14.2 What each interface looks like

Each interface defines a small surface — typically 3–6 methods — that the rest of the system depends on. Implementations live behind feature flags, swappable per-tenant or globally. Example:

```
interface LLMProvider {
  complete(prompt: string, opts: CompletionOpts): Promise;
  stream(prompt: string, opts: StreamOpts): AsyncIterable;
  embed(text: string): Promise;
  judge(input: any, criteria: string[]): Promise;
}
```

14.3 The discipline

- **No vendor SDK is imported outside its provider implementation file.** If import Anthropic from '@anthropic-ai/sdk' shows up anywhere except providers/llm/anthropic.ts, the PR is rejected.
- **Configuration is per-tenant.** A tenant on a beta voice provider doesn't affect other tenants.
- **Telemetry is at the interface, not the implementation.** Latency, cost-per-call, error rate are tracked in interface terms — so swapping implementations preserves the dashboards.

WHY THIS COMPOUNDS

When Sonnet 5 ships at 30% lower cost with better instruction-following, you change one line. When ElevenLabs raises prices, you swap to Twilio ConversationRelay tenant by tenant. When Meta deprecates a WhatsApp template format, the messaging provider absorbs the change. The Karpathy framing again: *the harness ships even if the model changes underneath.*

PART IV

Build methodology

How v3.0 actually gets built at 4–6 hours per week. The bottleneck is not your typing speed; it's your decision speed. The methodology optimises for that.

15. Claude Code + Codex operating model

OPERATING PRINCIPLE

The tools write the code. You write the spec, open the accounts, review the PRs, run the test calls. Every hour you spend in a text editor is an hour Claude Code or Codex should have spent there instead.

15.1 Division of labour

Task	Owner	Tool
Write the v3.0 specification	Lukasz	This document + BUILD_LOG.md
Open vendor accounts, do KYC, get API keys	Lukasz	Browser, ID documents
Decide stack trade-offs when ambiguous	Lukasz	Slack-with-self in BUILD_LOG
Scaffold the repo (Next.js, Fastify, TypeScript, lint, tests)	Claude Code	Terminal session w/ Lukasz watching
Postgres schema migrations	Claude Code	Terminal session
MCP server implementations	Claude Code	Terminal session
Provider interface implementations	Claude Code	Terminal session
Test harness, golden dataset loader, eval runner	Claude Code	Terminal session, paired with Lukasz on labelling
Voice ConvAI agent provisioning script	Claude Code	Terminal session
Inbox UI	Codex (or Claude Code)	Parallel work, smaller scope
Synthetic check infrastructure	Codex (or Claude Code)	Parallel work, well-scoped
Hand-label 50 golden dataset examples	Lukasz	Pen, paper, Terry's recordings
Cutover Terry's number on day 56	Lukasz	Twilio console, mobile phone
Outbound & content during the build	Lukasz	WhatsApp, LinkedIn

15.2 How a build night actually runs

A typical Monday or Tuesday evening (2.5 hours):

1. **10 min — open BUILD_LOG.md.** Read last week's 'next action.' Confirm it's still the right next thing.
2. **15 min — write tonight's spec.** One paragraph: the goal, the inputs, the expected output, the definition of done. Paste into Claude Code as the kickoff prompt.
3. **90 min — paired session with Claude Code.** Claude Code writes; you review every non-trivial change; you run the tests; you push when green. If a decision needs human judgment, you make it and continue.
4. **15 min — write what happened in BUILD_LOG.** What shipped. What broke. Definition of next session's 'next action.'
5. **20 min — closing review.** Skim the diff. Make sure no secrets are committed. Make sure the deployed app still passes /health. Close the laptop.

15.3 When to use Codex vs Claude Code

Both are useful. They're different tools for different shapes of work.

Shape of work	Better tool	Why
Greenfield architecture, ambiguous requirements	Claude Code	Better at reasoning across the whole system, asking clarifying questions, weighing trade-offs.
Well-defined task, narrow scope, clear definition of done	Either — Codex is faster	Codex shines when the spec is tight.
Bug hunt across multiple files	Claude Code	Better long-context reasoning.
Refactor a single module to match an interface	Codex	Tighter loop, cheaper.
Eval harness, test data, regression suite	Claude Code	Subtle, requires understanding the data.
UI components from a screenshot or description	Codex	Faster iteration on visual stuff.

RULE OF THUMB

If you can write the spec in 2 sentences, hand it to Codex. If the spec needs more than that, hand it to Claude Code. If you can't write the spec in 5 sentences, the spec isn't ready — sleep on it.

16. Lukasz's actual job

What you do every week. Five activities. None of them are typing code into an editor.

16.1 Sign-ins, KYC, account ops

Several vendors require human verification with ID, business documents, or a decision-maker phone call. Tools cannot do this for you. Front-load it.

- Open accounts (week 1): Render, Postgres, Clerk, Stripe, Twilio, ElevenLabs, Anthropic, Sentry, Axiom, UptimeRobot, Meta Business Manager.
- Submit Meta WABA verification (week 1) — the 5–10 day clock starts immediately.
- Buy Twilio number for Mr. Lobster main line (week 1).
- Stripe Tax setup (week 4).
- ICO data protection registration (week 6) — £40/year, 20-minute online form.

16.2 Decision-making

Claude Code will surface dozens of trade-offs. Most of them are easy. Some are not. When in doubt, the test is: 'does this decision affect Terry's 8-week ship date?' If no — pick the one that's simpler and move on. If yes — sleep on it, then write the answer in `BUILD_LOG.md`.

16.3 Spec writing

Each session opens with you writing 5–10 sentences of spec for Claude Code. This is where your VFX-pipeline brain compounds. You've been writing precise specifications for systems for 17 years; this is the same skill applied to a different medium.

A good spec for Claude Code includes:

- The goal in one sentence.
- The inputs (data shapes, API surfaces, files).
- The outputs (data shapes, files, side effects).
- The constraints (must use this provider interface, must not block on this dependency).
- The definition of done (a specific test that passes; a specific behaviour observable).

16.4 Review and test

You don't need to read every line Claude Code writes — that defeats the point. You do need to:

- Read the diff summary.
- Confirm the test it claims to pass actually passes (run it yourself).
- Look for secrets in commits.
- Spot-check the parts of the system you care about most (data shapes, provider boundaries, tenant isolation logic).

16.5 Outbound & content (the parallel track)

The 4–6 hours/week build budget is one track. Outbound, content, and the VFX database run on the other tracks per your existing weekly rhythm. During the 8 weeks of v3.0:

- **Mon evening:** Mr. Lobster outbound (5 messages) — half cadence during build.
- **Tue evening:** v3.0 build night #1.
- **Wed evening:** Open. Outbound or content.
- **Thu evening:** One LinkedIn post + one X post about the build. Document this 8-week arc in public — it will be the best content you ever publish.
- **Fri evening:** Mr. Lobster outbound (5 messages) — half cadence.
- **Sat morning:** Optional second build session if energy is there.
- **VFX database:** 6–7pm at work continues unchanged. That's a separate track and compound interest.

17. The spec → code → test loop

One pattern, repeated for every feature. Run it cleanly and you ship. Run it sloppily and you accumulate debt that breaks at customer #3.

17.1 The four-step loop



17.2 BUILD_LOG.md as system of record

One file in the repo root. Append-only. Every session adds one entry. Format:

```
## 2026-05-04 (Mon, 21:00-23:30)

**Goal:** Wire Twilio webhook gateway with signature verification.

**Did:**
- Created /api/v1/twilio/incoming endpoint
- Verified signatures in middleware (helpers/twilio-sig.ts)
- Added dedupe by call SID
- Persisted raw payload to events table
- Tested with synthetic webhook (saved as test/fixtures/twilio.json)

**Decisions:**
- Picked pg-boss over BullMQ for queue (Postgres-native, no Redis to operate)
- Used jsonb for raw payload (queryable later)

**Broke / risks:**
- Signature verification depends on TWILIO_AUTH_TOKEN being set; CI will need this

**Next action:**
- Wire ElevenLabs ConvAI tool gateway (mirror image of Twilio, but for log_enquiry
)
```

WHY THIS MATTERS

The BUILD_LOG is what lets you pick up next Tuesday at 21:00 without spending 30 minutes remembering where you were. It's also content — the most authentic 'build in public' material is just polished BUILD_LOG entries. And it's a defensive document: if something goes wrong with Terry six months from now, you can trace exactly when and why a decision was made.

PART V

Terry v3.0 launch plan (8 weeks)

The only thing this part does: get Terry on v3.0 in 8 calendar weeks at 4–6 hours of build time per week. Eight weekly milestones, each one a defensible stopping point.

18. Week-by-week build sequence

Week 1 — Accounts, scaffolding, deploy

Outcome: A deployed health endpoint at `api.mrlobster.co.uk` on Render. WABA verification submitted. Eleven accounts opened.

- **Day 1 (Lukasz, ~2h):** Open every account in §22. Submit Meta WABA verification (business documents, privacy policy, use case statement).
- **Day 2 (Claude Code session, ~2h):** Scaffold Next.js + Fastify TypeScript repo. Lint, format, test setup. Deploy to Render. Wire `api.mrlobster.co.uk`. Health endpoint returns OK.
- **Day 3 (Lukasz, ~30min):** Write `BUILD_LOG.md` and `DOCS/decisions.md`. Set scope discipline.

Week 2 — Multi-tenant database foundation

Outcome: Terry is a tenant row. Config-as-data is real.

- Postgres on Render, migrations infrastructure (drizzle or kysely).
- Tables: `tenants`, `business_profiles`, `channels`, `service_catalogue`, `capture_fields`, `escalation_rules`, `agent_events`, `agent_tool_calls`, `audit_logs`.
- Row Level Security policies (`tenant_id` required on every domain row).
- Seed Terry from his current ElevenLabs SOUL.md content.
- `getTenantConfig(tenantId)` helper. **One command loads Terry's brain.**

Week 3 — Provider abstractions + MCP scaffolding

Outcome: The five provider interfaces exist. The MCP host is scaffolded.

- `providers/llm/anthropic.ts`, `providers/voice/elevenlabs.ts`, `providers/messaging/twilio.ts`, `providers/telephony/twilio.ts`.
- MCP host using the official TypeScript SDK. First two MCP tools: `get_business_rules`, `get_service_catalogue`.
- Telemetry middleware: every provider call writes latency + cost to `usage_records`.

Week 4 — Webhook gateway + queue + persist worker

Outcome: A real Twilio test call flows end-to-end into Postgres. Owner alert sends.

- `/api/v1/twilio/incoming` with signature verification + dedupe.
- `/api/v1/elevenlabs/log_enquiry` as an MCP tool, signed.
- pg-boss queue. Persist worker (writes lead). Notify worker (sends Twilio WhatsApp).
- Synthetic test call: `ring` → Twilio → ConvAI handoff (basic agent) → `log_enquiry` → DB row → owner alert. End-to-end in <60s.
- Sentry receiving exceptions. Axiom receiving structured logs. UptimeRobot pinging `/health`.

Week 5 — Reception Agent SOUL + remaining MCP tools + Inbox UI

Outcome: Reception Agent works on Terry's data. Read-only Inbox shows leads.

- Compose Terry's SOUL from rows (services, rules, tone). Provision ElevenLabs agent via API.

- Remaining MCP tools: search_faq, check_escalation_triggers, estimate_job_value, create_lead, notify_owner, send_sms_fallback.
- Context engineering: structured-slots vs sliding-window pruning logic.
- Inbox UI: leads table, search, filter, export. Read-only. Mobile-friendly.

Week 6 — Golden dataset + eval harness

Outcome: 50 examples labelled. Eval runs on every captured lead.

- **Lukasz:** hand-label 30 examples from Terry's recordings (afternoon session).
- **Claude Code:** generate 20 synthetic adversarial examples (rambling caller, gas-smell mid-call, postcode confusion, etc).
- Field-level eval runner. Block-deploy-on-regression GitHub Action.
- score_eval MCP tool runs on every captured lead. Pass rate visible in admin dashboard.

Week 7 — Reliability hardening

Outcome: The system pages Lukasz before Terry notices anything is wrong.

- Reconciliation job (nightly Twilio call log API vs DB).
- Synthetic test-tenant fires end-to-end every 5 minutes.
- P1/P2 alerting routes (WhatsApp + SMS to Lukasz on synthetic failure).
- Voice failover: ElevenLabs ws drop → voicemail → Whisper transcribe → surfaced as lead.
- WABA verification expected complete. Migrate Terry's WhatsApp from Twilio to Meta direct.

Week 8 — Pre-go-live, cutover, day 1 of Terry on v3

Outcome: Terry is live on v3. Lukasz has not been to Chelmsford.

- **Day 1 — Saturday pre-flight:** Run the full pre-go-live checklist (\$19). Any failure pushes go-live to next week.
- **Day 2 — Sunday rollback test:** Confirm old AWS path can be reattached in <2 minutes. Take screenshots.
- **Day 3 — Monday cutover (~30min, evening):** One TwiML change on Terry's Twilio number to point at the new ConvAI bridge. Test call. Confirm.
- **Day 4–7:** Watch the dashboard. Triage any anomalies. Compare lead capture against pre-Mr.-Lobster baseline. Write the case-study seed entry in BUILD_LOG.

WEEKS 9–10 — SOFT HARDENING

Not part of the 'ship Terry' sprint. After day 56, two more weeks of non-feature work: address whatever the first week of real Terry traffic surfaces, polish the Inbox UI, write the customer-facing onboarding script for customer #2.

19. Pre-go-live checklist & cutover

19.1 The Saturday must-pass test (week 8 day 1)

Every item must pass. Any failure pushes go-live by one week. The cost of a delay is days. The cost of a second failed install is the relationship.

- **1.** Synthetic call: customer dials → Twilio → ConvAI → log_enquiry → DB row written → owner WhatsApp received within 60s.
- **2.** Synthetic call with simulated ElevenLabs failure: voicemail captured, transcribed by Whisper, surfaced as a normal lead.
- **3.** Synthetic call with simulated Meta WhatsApp failure: SMS fallback delivered.
- **4.** Synthetic call with mid-conversation gas-smell trigger: escalation alert fires within 5s of the trigger word, NOT after the call ends.
- **5.** Reconciliation job: any call in Twilio's API not in DB triggers a P1 alert. Tested by deleting a row.
- **6.** Terry's ElevenLabs agent has exactly the configured MCP tools attached. All four old deprecated tools confirmed deleted by API query.
- **7.** Hard cap on call duration set (4 min). Tested with a long synthetic call.
- **8.** 30s silence hangup tested.
- **9.** Sentry receiving exceptions with tenant_id. Axiom showing structured logs with correlation IDs. UptimeRobot pinging every endpoint.
- **10.** Eval harness: golden dataset of 50 examples runs in CI. Block-on-regression confirmed with a deliberate bad commit.
- **11.** Rollback tested: old AWS path can be reattached to the Twilio number with one TwiML change in <2 minutes.
- **12.** WABA verification confirmed (or fallback plan: continue on Twilio WhatsApp for owner alerts only).
- **13.** Terry has been told the time of cutover and has confirmed his phone is available for the test call.

19.2 Cutover plan (week 8 day 3)

Total time: 30 minutes. Lukasz at his laptop. Terry on the phone (informed, not physically present).

- 1. 00:00** — call Terry, confirm he's available for a test call in 10 minutes.
- 2. 00:05** — change the TwiML on Terry's existing Twilio number from old AWS endpoint to the new ConvAI bridge.
- 3. 00:08** — Lukasz dials Terry's number from a different phone. Lets it ring 20s without Terry picking up. ConvAI takes the call.
- 4. 00:11** — speak through a synthetic enquiry (boiler service, postcode, name).
- 5. 00:13** — confirm Terry receives the WhatsApp alert with the structured details.
- 6. 00:15** — confirm the lead appears in the Inbox UI.
- 7. 00:20** — call Terry back, confirm he received the alert correctly. Tell him he's live.
- 8. 00:30** — write the BUILD_LOG cutover entry. Take a screenshot of the first real lead. (This becomes content.)

ROLLBACK CONDITIONS

Any of: (a) ConvAI doesn't pick up the test call, (b) the WhatsApp alert doesn't arrive within 60s, (c) the lead doesn't appear in the Inbox, (d) Terry isn't comfortable. Action: change the TwiML back to the AWS endpoint (2 minutes). Continue triage offline. Reschedule cutover within 7 days.

PART VI

6-month roadmap

Where v3 goes after Terry is live. Three more product phases (v3.1, v3.2, v3.3) and the customer acquisition rhythm that runs in parallel.

20. v3.0 → v3.1 → v3.2 → v3.3

Phase	Timing	Headline capability	Customer-facing
v3.0	Weeks 1–8 (May–June)	Reception Agent, lead capture, owner alerts.	'Mr. Lobster — never miss another job.'
v3.0a	Weeks 9–10	Hardening. Onboard customers 2–3 manually.	Same.
v3.1	Months 3–4 (July–August)	Self-serve onboarding wizard. Stripe billing. Inbound WhatsApp. Customer identity + history. Optional booking. Owner Assistant Agent (basic).	'Mr. Lobster — answers, books, and remembers.'
v3.2	Months 5–6 (September–October)	Memory Agent. Growth Advisor Agent. Conversational config via WhatsApp. Monday digest. Self-building setup from website + Google Business.	'Mr. Lobster — your company agent.'
v3.3	Post-month 6	Vertical agents (Plumber, Heating Engineer, Clinic). Outcome-based pricing tier. Public API. Web chat / Instagram DM channels.	Vertical-specific positioning per market.

20.1 v3.1 — Self-serve + Owner Assistant (months 3–4)

- Six-step onboarding wizard. Customer #4+ provisions themselves in <30min.
- Stripe Billing: setup fee + monthly subscription + metered overage.
- Meta Cloud API for new tenants. Twilio WhatsApp retired for new signups.
- Inbound WhatsApp customer conversations enabled (Claude Sonnet replies using tenant SOUL).
- Customer identity layer (phone / WhatsApp ID indexed across channels).
- Owner Assistant Agent on dedicated Mr. Lobster HQ WhatsApp number.
- Optional Google Calendar booking (off by default; tenant toggles on).
- Goal: 8 paying customers, ~£2,400 MRR.

20.2 v3.2 — Company Agent (months 5–6)

- Memory Agent (background) — extracts signals, proposes memory updates, never commits without owner approval.
- Growth Advisor Agent — Monday digest, follow-up suggestions, pricing-gap detection, missed-revenue analysis.
- Conversational config via WhatsApp HQ — owner edits prices, rules, hours by sending messages.
- Self-building onboarding — scrape tenant website + Google Business + reviews to auto-populate config.
- SOUL auto-calibration — nightly drift detection against structured tables.
- First case study published. First public uptime status page.
- Goal: 15 customers, ~£4,500 MRR. First customer churn analysed (if any).

20.3 v3.3 — Verticals + scale (post-month 6)

- Plumber Agent (Terry's vertical, productised).

- Heating Engineer Agent.
- Clinic Reception Agent (multi-practitioner, GDPR-tight, appointment-aware).
- Estate Agent Enquiry Agent.
- Outcome-based pricing tier launched (with cohort data from v3.2).
- Public API + MCP server registry — third-party integrations.
- Web chat widget. Instagram DM channel.
- Goal: 30–40 customers, ~£9k–£12k MRR, gross margin 65–70%.

21. Customer acquisition rhythm alongside the build

The build does not absorb the founder. The existing weekly rhythm continues; outbound just runs at half-cadence during weeks 1–8.

21.1 Outbound during the v3.0 build (weeks 1–8)

- Mon evening: 5 WhatsApp messages.
- Fri evening: 5 WhatsApp messages.
- Total: 10/week × 8 weeks = 80 messages.
- Target: 1 demo booked by week 6. Customer #2 on a soft-pencil for week 10 onboarding.

21.2 Outbound after v3.0 (weeks 9+)

- Resume full cadence: Mon 5, Wed 5, Fri 5.
- Now leading with the Terry case study (specific numbers).
- Targets refined per existing roadmap: estate agents, dog groomers, mobile mechanics, multi-practitioner clinics, financial advisors.
- Goal by month 3 end: 8 paying customers.

21.3 Content during weeks 1–8

The build itself is the best content arc you've ever had. Document it on Thursday evenings:

- **LinkedIn (Thu):** One post per week. The build journey, the gnarly decisions, the Karpathy/Software-3.0 framing applied to a real local business problem. AI-native VFX founder positioning compounds.
- **X / Twitter (Tue + Thu + Sat):** 3 posts/week. Screenshots of BUILD_LOG. Real numbers. What broke. Build-in-public is the engine.
- **Instagram (1–2 posts/week):** Lobster Time clips. Behind-the-scenes of the build. Targeting tool, not primary outreach.

THE 8-WEEK CONTENT ARC

Week 1: "I'm rebuilding Mr. Lobster from scratch in 8 weeks. Here's the plan." Week 4: "Halfway. Here's what I got wrong in v1 and v2." Week 8: "Terry just went live on v3. Here's what happened on day 1." This arc, posted across LinkedIn, X, and Instagram, is the highest-leverage marketing you'll do all year. It's also free.

PART VII

Stack, accounts, costs

What you're paying for. What you sign up for. What it costs to serve a customer.

22. The exact account list

Open all of these in week 1, day 1. Some take days for verification (Meta WABA, Stripe). Front-loading them removes them from the critical path.

Vendor	Plan / cost	What for	Lukasz needs
Render	Starter \$7/mo (Postgres + web service starter combo ~\$30/mo)	Hosting, managed Postgres	Email, payment card
Clerk	Free tier (10k MAU)	Auth (2FA, password reset, sessions)	Email
Stripe	Standard (1.5% + 20p per UK card)	Subscriptions, setup fees, metered usage	Business details, bank account, ID
Twilio	Pay-as-you-go (~£0.012/min voice, £1/mo per number)	UK voice numbers, WhatsApp interim, SMS fallback	Email, payment card, business address
ElevenLabs	Creator \$22/mo (or higher; ConvAI usage on top)	Voice AI for Reception Agent	Email, payment card
Anthropic API	Pay-as-you-go (Sonnet \$3/M input, Haiku \$0.25/M input)	LLM for routing, eval, owner agent (v3.1+)	Email, payment card
Meta Business Manager	Free	Owns the WABA	Personal FB account, business documents, business website, privacy policy
Sentry	Team plan \$26/mo	Error tracking with tenant context	Email, payment card
Axiom (or BetterStack)	Free tier (5GB/mo) or paid \$25/mo	Structured logs, queryable	Email
UptimeRobot	Free tier (50 monitors, 5min interval)	Endpoint uptime + synthetic checks	Email
GitHub	Free / Pro	Source control, Actions for CI	Already have
UK ICO	£40/year	Data protection registration (legal requirement)	Personal details, business name

Total recurring cost (v3.0): ~£100/month for infrastructure (excluding usage). **Usage costs scale per customer** (~£96/customer/month at Standard tier — see §25).

FRONT-LOAD MENTAL NOTE

Meta WABA verification can take 5–10 days. Stripe live-mode activation can take 1–2 days for UK sole-trader. ICO registration is <1 day. Do all three in week 1 day 1, before any code is written. The build does not block on these; they verify in parallel.

23. Stack rationale

Layer	Choice	Why this, not the alternative
Hosting	Render (or Railway)	Solo founder needs zero DevOps. Push-to-deploy. Managed Postgres included. AWS at 100+ tenants if economics demand.
Backend	Node 20 + TypeScript + Fastify	Same language as Twilio/Meta SDKs. TypeScript prevents the data-shape bugs that killed v1. Fastify over Express for performance and built-in schema validation.
Database	Postgres (managed, on Render)	Boring, stateful, multi-tenant via tenant_id + RLS. Same DB hosts the queue.
Queue	pg-boss	Postgres-native. No Redis to operate. Job state queryable in SQL. Move to BullMQ + Redis at 100+ tenants if needed.
Auth	Clerk	Don't build auth. 2FA, password reset, sessions, organisation accounts free.
Telephony	Twilio Programmable Voice	UK numbers. Mature webhooks. Wrapped in TelephonyProvider for swap.
Voice AI	ElevenLabs ConvAI	Best UK voice quality today. Wrapped in VoiceProvider. Twilio ConversationRelay is the swap-in if pricing or reliability degrades.
WhatsApp	Twilio (interim) → Meta Cloud API	Faster to ship; cheaper at scale. Migration is invisible to tenant.
LLM	Anthropic (Sonnet + Haiku)	Sonnet for replies (quality, instruction-following). Haiku for routing/eval (cheap).
Frontend	Next.js (in same repo)	Same TypeScript codebase. Server components for data fetching.
Observability	Sentry + Axiom + UptimeRobot	Errors with tenant context, structured logs, synthetic uptime. £50/mo total.

WHAT IS NOT IN THE STACK — AND NEVER WILL BE

AWS EC2 + PM2 + nginx (the v1 mistake). WhatsApp Web / unofficial APIs / browser-emulating transports. Per-customer servers. Per-customer prompt files edited by hand. Mac Mini fallback. Tailscale across production. SSH-as-deploy. OpenClaw running customer traffic. Vendor-locked SDKs imported outside provider files.

24. Pricing & outcome-based hooks

The v2 documents settled on £149/£299/£499 with a £599 setup fee. v3 keeps that structurally but adds outcome data from day one so the v3.3 outcome tier becomes available.

24.1 v3.0–v3.2 pricing

Tier	Price	Included	Variable cost	Gross margin
Setup	£599 one-time	Onboarding, number, agent, DPA	~£20 in time	~£579
Starter	£149/mo	200 AI-handled calls	~£40	~£109 (73%)
Standard	£299/mo	600 AI-handled calls	~£96	~£203 (68%)
Pro	£499/mo	1,500 AI-handled calls	~£220	~£279 (56%)
Overage	£0.40/call beyond included	—	~£0.18	~£0.22 (55%)

24.2 Margin protection rules (non-negotiable)

1. Hard cap on call duration: 4 minutes. Stops runaway costs.
2. ElevenLabs already discounts long silences 95%. Still hang up after 30s of silence.
3. Per-tenant monthly cap: soft at tier limit, hard at 2× tier. Beyond 2×, AI takes name + number only (no full conversation).
4. WhatsApp reply length cap: 200 tokens. Multi-turn capped at 20 turns.
5. Usage alerts at 50% / 80% / 100% of tier.
6. High-volume tenants are auto-prompted to upgrade after 2 consecutive months in overage.

24.3 v3.3 outcome-based hook (cohort data permitting)

With three months of job_value_estimate + lead_outcomes data per cohort, two new pricing variants become testable:

Variant	Structure	Why this works
Performance	£99 base + 3% of estimated revenue protected, capped at £599/mo.	Lower friction to sign. Aligned incentives. Best-of-both for the mid-volume tenant.
Premium agent	£499/mo flat. Includes Company Agent, weekly digest, growth advisor.	The customers who want the AI relationship pay for it. Strongest retention.

DO NOT CHANGE PRICING TODAY

v3.0 ships at £299 Standard. v3.3 introduces the outcome variant with real data backing it. Don't guess at performance pricing without a cohort. Don't under-price the agent layer — if it advises, follows up, and reduces admin, it competes with part-time front-office help, not chatbot software.

25. Cost model per customer

25.1 Variable cost — Standard tier (~600 calls/month)

Cost line	Source	Monthly
ElevenLabs voice	\$0.088/min after 250 included min	~£50
LLM during voice	ElevenLabs ~\$0.029/call equivalent	~£14
Anthropic for WhatsApp replies	Haiku, ~£0.005/conversation	~£3
Twilio voice inbound	~£0.012/min	~£12
Twilio number rental	£1/mo per tenant	£1
WhatsApp (Meta utility)	£0.025 × beyond 1,000 free	~£5
Hosting amortised	£300/mo Render ÷ 50 tenants	~£6
Stripe processing	1.5% + 20p	~£5
Total variable cost		~£96

25.2 Cohort projection (illustrative, not a forecast)

Month	Customers	MRR	Setup (one-time)	Total
1	1 (Terry)	£299	£599	£898
3	5	~£1,500	~£2,400	~£3,900
6	15	~£4,500	~£3,600	~£8,100
12	40	~£12,000	~£3,000	~£15,000

By customer 15: revenue covers tools, infrastructure, and a part-time VA. By customer 30–40: a defensible position. The moat is operational maturity. Every customer who churns from a competitor finds Mr. Lobster actually works on Saturday mornings.

PART VIII

Reliability, security, GDPR

Inherited largely unchanged from v2 master plan, refined where the v3 architecture demands it.

26. Reliability plan

PRINCIPLE

If a call reaches Twilio, the owner gets a WhatsApp alert within 90 seconds — or someone is paged. There is no third option.

26.1 Retry policy per layer

Layer	Behaviour
Twilio → Gateway	Twilio retries 5× over 3h (built-in). Gateway dedupes by call SID.
ElevenLabs → MCP tool	Gateway returns 200 even on internal failure. Job enqueued. Failures retry from queue.
Persist to DB	5 retries, exponential backoff (1s / 4s / 16s / 1m / 5m).
Notify owner via Meta WA	5 retries with backoff. Then fallback to SMS via Twilio.
SMS fallback fails	Email to owner. Internal P1 alert to Lukasz.
Meta inbound	Meta retries automatically. Gateway dedupes by message ID.

26.2 Failover by layer

Failure mode	Behaviour
ElevenLabs outage	Twiml detects ConvAI websocket failure within 2s. Plays 'We're having trouble — please leave your name and number.' Voicemail recorded. Whisper transcribes. Surfaced as a normal lead.
Meta WhatsApp outage	Worker detects 5xx after 3 retries. Falls back to Twilio SMS to owner with same payload.
Database outage	Managed Postgres failover. Worker queues survive ~30s in-memory. Longer outages return 503s; Twilio retries.
Whole platform outage	Twilio number falls through to owner's mobile after timeout. Worst case = the customer's experience before Mr. Lobster existed.

26.3 Reconciliation (the safety net)

- Every call gets a row in the database the moment Twilio fires the FIRST webhook — even if processing later fails.
- Nightly job compares Twilio's call log API to the database. Any call in Twilio not in DB → P1 page.
- Any call in DB without an owner notification within 5 min → P1 page.
- The customer never has to tell us something broke. The system tells us first.

26.4 Synthetic checks & alerting

- Every 5 min, a test tenant fires a synthetic webhook end-to-end through the system. Latency tracked. Failure pages Lukasz on WhatsApp + SMS.
- Daily synthetic call (real Twilio dial-in to a test number) verifies the voice path.
- P1: synthetic check fails 2× in a row → WhatsApp + SMS to Lukasz. Escalate after 5 min.
- P1: any tenant has 0 webhooks in last 1h during business hours → WhatsApp.
- P2: notify worker fails >5% over 10 min → WhatsApp.
- P2: Sentry error rate spike → email digest, hourly.
- P3: single tenant nearing usage limit → email tenant + Lukasz.

27. Security & GDPR

Mr. Lobster is a data processor; tenants are controllers. The minimum-to-operate-legally list:

27.1 Required from day 1

- DPA signed with every tenant at onboarding (Stripe Checkout step from v3.1).
- Privacy policy listing sub-processors: Twilio, ElevenLabs, Meta, Anthropic, Render.
- Tenant-facing privacy notice template the customer publishes on their own site.
- UK ICO registration (~£40/year for sole trader). 20-minute online form.

27.2 Data lifecycle (v3.0 defaults)

Data type	Default retention
Call audio recordings	30 days, then deleted. Configurable per tenant: 0 / 30 / 90.
Call transcripts	12 months, then anonymised.
Lead records	Until tenant deletes, or 24 months after last contact.
WhatsApp message bodies	12 months.
Operational logs	30 days.
Backups	Encrypted. 30 days.
Audit logs	12 months.

27.3 Encryption

- TLS 1.2+ everywhere. Webhook signatures verified before processing.
- Postgres disk-level encryption at rest (host default).
- Sensitive fields encrypted with libsodium. Keys in env, rotated quarterly.
- No API keys in code. No tokens in chat / Telegram messages. Secrets in host's secret manager.

27.4 Access control

- Tenant auth: Clerk. Email + password + optional 2FA.
- Roles: Owner / Manager / Engineer (v3.0 ships Owner only; roles in v3.1).
- Admin (Lukasz): separate admin app, hardware key required, every cross-tenant query logged.
- API keys: none exposed to tenants in v3.0–v3.1. All vendor calls server-side.

27.5 Audit log

Every state change tracked: who, what, when, from where. Append-only audit_logs table. Surfaced as 'Activity' in the dashboard. SOC 2 readiness from customer ~25.

27.6 Incident response

Documented breach playbook: identify → contain → assess → notify ICO within 72h if personal data affected → notify tenants → notify their customers if required. Retainer with a UK GDPR consultant (~£200/month from customer 20).

Risks & mitigations

Risk	Severity	Mitigation
Terry's WhatsApp WABA verification fails or stalls	High	Use Twilio WhatsApp for owner alerts on day 56 (no template approval delay). Verification runs in parallel from week 1. If Meta declines, Twilio is the indefinite fallback.
ElevenLabs reliability or pricing changes	High	VoiceProvider abstraction from day 1. Twilio ConversationRelay wired in as a swap-in alternative. Monitor; switch tenant-by-tenant if needed.
Single noisy customer blows up unit economics	High	Hard caps: 4 min/call, 30s silence hangup, 2× tier monthly cap, name+number-only mode beyond cap. Auto-upgrade prompts.
Cutover fails on day 56	Critical	No site visit. Cutover is one TwiML change, done remotely. Pre-go-live checklist must pass. Rollback tested. Old AWS path stays warm 7 days.
AI hallucinates prices or jobs	Medium	Agent quotes from structured services table only. Negative constraints in SOUL. Capture-only default — no booking until tenant explicitly enables.
GDPR breach	Medium	DPA, privacy policy, ICO registration, retention defaults, encryption, audit log, breach playbook. £200/mo GDPR consultant from customer 20.
Support burden on Lukasz scales linearly	Medium	Self-serve onboarding by month 3. Conversational support via WhatsApp from month 5. Knowledge base from month 4. Hire VA for setup support from customer 15.
WhatsApp template rejections or policy changes	Low–Medium	Templates kept conservative (utility category). Service-conversation pricing (free within 24h customer-initiated window) used where possible. SMS fallback always available.
Render or hosting provider outage	Low	Multi-region within Render is available. Worst case: Twilio falls through to owner mobile = pre-Mr.-Lobster baseline.
8-week timeline slips	Medium	Acceptable to slip by 1–2 weeks if quality is preserved. Unacceptable to ship a second failed install. Default is to delay rather than push through.
Lukasz over-commits across Mr. Lobster + VFX Database + content	High (founder)	8-week build is the bound, not a sprint. Outbound at half-cadence weeks 1–8. VFX Database 6–7pm sessions continue. BUILD_LOG enforces stop-times.
Distraction by L3 / Company Agent features during v3.0	High (founder)	Discipline: L2 is v3.1, L3 is v3.2. Every 'could we add...' question during weeks 1–8 defaults to NO. Backlog it. Ship the boring reliable thing first.
Eval discipline lapses after Terry is live	Medium	Eval runs on every captured lead, not just on prompt change. Block-deploy-on-regression GitHub Action. Monday review of eval scores is a calendar item, not a 'when I have time' activity.
MCP standard changes / fragments	Low	MCP host is a single file. Refactor cost is hours, not weeks. Worth the bet given the upside.

PART X

Founder action plan

What you do tonight, this weekend, and across the eight weeks.

28. Next 48 hours

Tonight (Thursday 30 April)

- 1. Reprice the website (15 min).** Remove three-product framing. One product: Mr. Lobster Reception Engine. Standard tier £299/mo. £599 setup. Voice + WhatsApp + lead capture as feature bullets, not products.
- 2. Tell Terry the plan (10 min, WhatsApp).** “We’re rebuilding on a stable cloud system over the next 8 weeks. No site visit. I’ll need 30 minutes of your time on day 56 for the cutover. Everything else I do remotely.”
- 3. Write the Friday content (20 min).** LinkedIn post: ‘I’m rebuilding Mr. Lobster from scratch in 8 weeks. Here’s the plan, and here’s what I got wrong in v1 and v2.’

Friday 1 May

- 1. Open the eleven accounts (~3h).** All of \$22. Front-load the verifications.
- 2. Submit Meta WABA verification (~30 min).** The 5–10 day clock starts. Use the framing from the addendum: “customer service and front-office automation platform for local service businesses, handling job enquiries, owner notifications, and customer support workflows.” Avoid ‘chatbot.’
- 3. UK ICO registration (~30 min, online).** £40. 20-minute form. Done.
- 4. Friday outbound (1h).** 5 WhatsApp messages, half-cadence rhythm.

Saturday 2 May (optional, if energy)

- Install Claude Code if not already installed. Test on a throwaway repo.
- Create the Mr. Lobster v3 GitHub repo, private, with a single README.md and BUILD_LOG.md.
- Write the v3 BUILD_LOG entry zero: copy the principles from this document into DOCS/decisions.md.

DO NOT WRITE CODE YET

Week 1 day 2 is when Claude Code starts scaffolding. Before that, your job is to open accounts and submit verifications. Writing code in week 0 means doing it twice.

29. Next 7 days & build backlog

29.1 The first build week (Mon 4 May → Fri 8 May)

- **Mon evening (Mr. Lobster outbound 5×):** WhatsApp messages, half-cadence.
- **Tue evening (Build night #1, ~2.5h):** Claude Code session. Outcome: deployed Next.js + Fastify TypeScript app on Render. /health returns OK. BUILD_LOG entry written.
- **Wed evening (Open):** Either continue build (postgres scaffolding) or rest.
- **Thu evening (Content):** LinkedIn post about the build. X thread: 3 posts. Lobster Time clip if available.
- **Fri evening (Mr. Lobster outbound 5×):** Half-cadence outbound.
- **Throughout the week:** WABA verification documents replied to as Meta requests them. Sentry receiving exceptions. Axiom showing logs. UptimeRobot pinging.

29.2 Build backlog (prioritised)

P0 — Must build now (v3.0, weeks 1–8)

- Multi-tenant Postgres schema (all v3.0 tables incl. eval_runs, memory_proposals, extracted_signals, job_value_estimate on leads).
- Twilio webhook gateway with signature verification + dedupe + raw payload archive.
- ElevenLabs MCP tool gateway — every tool an MCP server.
- pg-boss queue with persist + notify + memory_extract + eval workers.
- Provider interfaces: LLMProvider, VoiceProvider, MessagingProvider, TelephonyProvider.
- ElevenLabs agent provisioning via API (no dashboard clicks ever).
- Twilio number provisioning + TwiML attach.
- Twilio WhatsApp owner alerts.
- Inbox UI — read-only, with search and export.
- Admin dashboard for tenant lifecycle (Lukasz only).
- Reconciliation job (nightly).
- Synthetic check (every 5 min, end-to-end).
- Sentry + Axiom + UptimeRobot wired.
- Hard caps: 4 min call duration, 30s silence hangup.
- Context engineering: structured-slots vs sliding-window pruning.
- Golden dataset: 50 hand-labelled examples + field-level eval runner.

P1 — Build for v3.1 (months 3–4)

- Self-serve onboarding wizard (6 steps).
- Stripe Billing: setup fee + monthly + metered overage.
- Meta Cloud API integration (replaces Twilio WhatsApp for new tenants).
- Inbound WhatsApp customer conversations (Claude Sonnet replies).
- Customer identity layer (phone / WA ID indexed across channels).
- Conversation history per customer.
- Owner Assistant Agent on Mr. Lobster HQ WhatsApp.

- Calendar booking (Google OAuth, optional, per-tenant toggle).
- Persona sliders compose into SOUL automatically.
- Escalation rules engine refinement.
- DPA + privacy policy + tenant privacy notice template.
- LLM-as-judge eval layer.

P2 — Build for v3.2 (months 5–6)

- Memory Agent (background extraction).
- Growth Advisor Agent.
- Conversational config via WhatsApp HQ number.
- Self-building setup from website + Google Business + reviews.
- Weekly digest WhatsApp to owner.
- Follow-up automation.
- Owner approvals via WhatsApp.
- SOUL auto-calibration (system corrects toward structured tables).
- Public uptime status page.
- Multi-user roles (Owner / Manager / Engineer).
- Number portability (port-out for £25).

P3 — Later / future (v3.3+)

- Vertical agents (Plumber, Heating Engineer, Clinic Reception, Estate Agent).
- Outcome-based pricing tier.
- Public API + MCP server registry.
- Web chat widget channel.
- Instagram DM channel.
- SMS as a third channel.
- Voice-clone of the owner ('sounds like Terry').
- Per-tenant analytics dashboard with longitudinal insights.
- Marketplace of pre-built personas / capture flows by trade.
- Outbound campaigns ('WhatsApp my last 100 customers about a winter offer').
- Multi-language support.
- SOC 2 certification.

CLOSING

The face of AI revolution, locally

Karpathy ends his talk on a note about timing — that this is one of the rare moments in computing history when the ground actually shifts. Software 1.0 had its founders. Software 2.0 had its founders. Software 3.0 is being founded right now. The seats are still open.

Most of those seats will be taken by people building general-purpose AI tools for people who already use AI tools. The white space — and it is enormous — is in the segments that haven't shown up yet. UK local trades. Family-run clinics. The plumber who answers his phone with a wrench in his hand. The clinic manager who runs Instagram ads but still misses 40% of the calls those ads generate.

These aren't edge cases. These are the real economy. They're also the people for whom AI has been a story, not a tool. Mr. Lobster is what flips that.

THE COMMITMENT

By the end of October 2026 — six months from today — Mr. Lobster has 30 customers in Essex, London, and the home counties. Every one of them runs an AI receptionist that knows their business better than any tool a competitor could ship. Every one of them has stopped losing jobs. Every one of them has had at least one moment of looking at the Monday digest and thinking *this is what people mean when they talk about AI*.

That moment — multiplied across thirty plumbers, ten clinics, fifteen estate agents — is what makes Mr. Lobster the local face of the AI revolution. Not a marketing tagline. A literal description of who, in Essex, is using AI to run their business.

v1 was the toolkit. v2 was the architecture. v3 is the product. Build it boring, build it reliable, build it future-proof. Ship Terry. Ship customer 5. Ship the Company Agent. Ship vertical agents. Ship the outcome tier. Each step is a compound interest entry on the previous one.

v1 was the toolkit. v2 was the architecture. v3 is the product. The customer isn't buying AI. The customer is buying a front office that finally works on a Saturday morning.

Mr. Lobster v3 — Master Build & Vision Plan

Lukasz Bukowiecki · 30 April 2026

Generated as a synthesis of: Mr_Lobster_v2_Master_Build_Plan (26 Apr 2026), Mr_Lobster_v2_Update_Addendum (28 Apr 2026), and analysis informed by Andrej Karpathy's Software is changing again (2025).